IdPGroup

Identification functions for different p-groups

1.0

11 August 2026

Henrik Schanze

Henrik Schanze Email: henrik.schanze@tu-braunschweig.de
Homepage: https://www.iaa.tu-bs.de/henschan
Address: Universitaetsplatz 2

Contents

1 Introduction **3**

2 Functionality **4**

2.1 Identifying groups via a search tree 4

2.2 Loading (strong) siblings 6

2.3 Functions for calculating (strong) siblings 8

2.4 Functions for building a search tree 9

Index **15**

Chapter 1

Introduction

The purpose of the IdPGroup package is to implement identification functions for different p-groups and thereby extend the functionality of the SmallGrps library. Currently this includes an identification function for groups of order 512.

Details of the implementation and functionality of the identification functions are dicussed in section 2.1.

Section 2.2 introduces the definitions of (strong) siblings and includes functions to load all such sets of groups of certain order.

Functions that were used to calculate these lists of groups and which can be used to calculate them for different orders can be found in section 2.3.

Finally section 2.4 describes all funcions that were used to build the search tree in the descirbed identification routine and which can be used to build other such trees.

Chapter 2

Functionality

This chapter lists all functions contained in the IdPGroup package.

2.1 Identifying groups via a search tree

This section briefly describes the functions and variables of this package associated to the identification of an arbitrary group of order 512 via a search tree. By 'identify' a group G we mean that we want to find *id* such that `SmallGroup(512, id)` is isomorphic to G. This is done by calculating invariants and comparing them with the ones of the remaining candidates. This process is organized in the tree `ID_GROUP_512_TREE` (2.1.3).

The used tree is implemented as nested records with the following components:

- "fp": a list of integers, each being the encoded fingerprint of groups resulting from the specific test performed at the current level of the identification routine.
- "next": a list comprised of the respective subbranches corresponding to the fingerprints in "fp". This is either a list of ids or a branch implemented as a record itself.

Depending on the depth in the tree each record also has one of the following two components:

- "test": a string that indicates which test was used via `TestOnGroup` (2.4.2) to create the listed fingerprints.
- "desc": a list of integers that encode a description of the performed powermap-test

The first levels of `ID_GROUP_512_TREE` (2.1.3) try to distinguish groups by the following invariants:

- the rank of the group,
- the collected orders of all elements of the group,
- the numbers of conjugacy classes of each length and representative order (the order of the elements in the class) of the group,
- the isomorphism type of the parent of the group, i.e. the quotient of the group and the smallest non-trivial element of the lower-p-central series,

- the abelian invarants, i.e. the isomorphism types of the quotients of the derived series and
- the cycle structure of the permutations induced on all clusters of conjugacy classes of same length and representative order by the power map $x \mapsto x^3$.

If necessary these invariants are followed by invaraints resulting from tests evaluated on partitions of G. The initial partition is made of the conjugacy classes of G grouped such that conjugacy classes of the same length and same representative order belong to the same partitioning subset. The results of a power map applied to a partition give the invaraints used to distinguish groups in these levels of the search tree. The exact behaviour is determined by the content of the component "desc". After the evaluation of each test, the partition is refined according to the result.

If the preceding invaraints do not lead to a single group, the following (more time consuming) invaraints are calculated.

- The isomorphism types of the non-trivial factors formed by the group and elements of the smallest non-trivial element of the lower-p-central series,
- the isomorphism types of the non-trivial factors formed by the group and elements of the center and
- the isomorphism types of all maximal subgroups.

Remaining are 139 pairs and 1 triplet of groups of unidentified groups. All these groups are distinguished by a random isomorphism test the same way as groups of order 256 are finally distinguished in the SmallGrps library.

2.1.1 InfoId512

▷ `InfoId512` (info class)

Info class for the identification routine of groups of order 512.

2.1.2 IdGroup512

▷ `IdGroup512(`*`G[, opt]`*`)` (function)

Returns: The id of the group of order 512 that is isomorphic to `G`.

Identifies the group `G` of order 512. This function accepts a record `opt` of options with the following content:

- `tree`: A search tree that was built using the invariants resulting from *tests*.
- `anupq`: A boolean that determines if *IsPqIsomorphicPGroup* from the Anupq-Package should be used to find the to `G` isomorphic group among the remaining candidates.
- `path`: A boolean that determines if the path through the tree shall be recorded while identification and returned as well.

The default values of `opt` is `rec(anupq:=false, tree:=ID_GROUP_512_TREE, path:= false )`.

Example

```
gap> G := Image(IsomorphismPermGroup(SmallGroup(512, 1984)));;
gap> IdGroup512(G);
1984
gap> IdGroup512(G, rec(path := true));
[ 1984, rec( fp := [ 2, 8217, 5321 ], pos := [ 2, 188, 1 ],
        tests := [ "Rank", "ElmOrds", [ 40511009 ] ] ) ]
gap> H := Image(IsomorphismPermGroup(SmallGroup(512, 449)));;
gap> IdGroup512(H, rec(anupq := false));
449
gap> IdGroup512(H, rec(anupq := true));
449
```

2.1.3 ID_GROUP_512_TREE

▷ ID_GROUP_512_TREE (global variable)

The search tree of groups of order 512 that can determine the vast majority of groups. At the beginning only a small part is loaded. Other branches are loaded if necessary.

2.1.4 LoadTotalTree

▷ LoadTotalTree() (function)

Loads the entire search tree ID_GROUP_512_TREE (2.1.3).

2.1.5 IdGroup512Exceptions

▷ IdGroup512Exceptions() (function)

Returns: A list of bins of group-ids, that ID_GROUP_512_TREE (2.1.3) can not distinguish without random isomorphism tests.

Example

```
gap> L := IdGroup512Exceptions();;
gap> Collected(List(L, Size));
[ [ 2, 139 ], [ 3, 1 ] ]
gap> L[138];
[ 6249609, 6249610, 6249611 ]
```

2.2 Loading (strong) siblings

This section describes the functions used to load precalculated lists of (strong) siblings, i.e. groups that fulfil the following definitions:

- *subgroup-equivalent groups* are groups with a bijection between the sets of proper subgroups s.t. preimage and image are isomorphic,
- *factor-equivalent groups* are groups with a bijection between the sets of proper factors s.t. preimage and image are isomorphic,

- *siblings* are subgroup- and factor-equivalent groups, where the bijection of proper subgroups preservs conjugacy and maps the Frattini subgroup, lower and upper central series onto the respective counterparts,
- *strong siblings* are siblings that are also Brauer Pairs.

All (strong) siblings of orders dividing 512 together with some trivial cases are implemented in this package and are callable by the following functions.

2.2.1 SiblingsIds

▷ `SiblingsIds(`*`order[, indices]`*`)` (function)

Returns: A list of lists of ids of siblings of size *`order`*.

Loads and returns a list of ids of siblings of size *`order`* if available. Currently available sizes are:

- all orders of the form pq for p and q primes,
- all powers of 2 smaller than 2^9
- all powers of 3 smaller than 3^5
- all powers of a prime smaller than p^4 and
- all orders smaller than 256 except 60, 120, 160, 168, 180, 189, 192 and 240.

If no list is available for the provided order `fail` is returned. If a list *`indices`* is given, only the respective entries are returned.

Example

```
gap> SiblingsIds(101);
[ ]
gap> SiblingsIds(128);
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
gap> SiblingsIds(256, [1..3]);
[ [ 227, 228 ], [ 258, 260, 261 ], [ 845, 846 ] ]
```

2.2.2 Siblings

▷ `Siblings(`*`order[, indices]`*`)` (function)

Returns: A list of siblings of size *`order`*.

Loads and returns a list of siblings of size *`order`* if available. If no list is available for the provided order `fail` is returned. If *`indices`* are given, only the respective entries are returned.

2.2.3 StrongSiblingsIds

▷ `StrongSiblingsIds(`*`order[, indices]`*`)` (function)

Returns: A list of lists of ids of strong siblings of *`order`*.

Loads and returns a list of ids of strong siblings of *`order`* if available. Currently available sizes are:

- all orders of the form pq for p and q primes,
- all powers of 2 smaller than 2^9

- all powers of 3 smaller than 3^5
- all powers of a prime smaller than p^4 and
- all orders smaller than 256 except 60, 120, 160, 168, 180, 189, 192 and 240.

If no list is available for the provided order `fail` is returned. If a list *indices* is given, only the respective entries are returned.

Example

```
gap> StrongSiblingsIds(256);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ], [ 1741, 1742 ],
  [ 3678, 3679 ], [ 4154, 4157 ], [ 4155, 4158 ], [ 4156, 4159 ] ]
gap> StrongSiblingsIds(256, [1..3]);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ] ]
```

2.2.4 StrongSiblings

▷ `StrongSiblings(`*order[, indices]*`)` (function)

Returns: A list of strong siblings of *order*.

Loads and returns a list of strong siblings of *order* if available. If no list is available for the provided order `fail` is returned. If a list *indices* is given, only the respective entries are returned.

2.3 Functions for calculating (strong) siblings

This section describes all functions for calculating (strong) siblings of a given order.

2.3.1 CalculateSiblings

▷ `CalculateSiblings(`*order*`)` (function)

Returns: A list of lists of ids of groups of size *order* that are siblings.

Determines all siblings among the groups of size *order* by applying the tests `"MaxSubId"`, `"SpSubId"`, `"SubId"`, `"MaxFactId"` and `"FactId"` iterativly. The tests `"MaxSubId"` and `"MaxFactId"` are not necessary, but are efficient and eliminate some sets before applying the more costly tests.

Example

```
gap> CalculateSiblings(128);
Test 'MaxSubId' found following bins of size greater one:[ [ 2, 23 ], [ 3, 2 ] ]
Test 'SpSubId' found following bins of size greater one:[ [ 2, 22 ] ]
Test 'SubId' found following bins of size greater one:[ [ 2, 16 ] ]
Test 'MaxFactId' found following bins of size greater one:[ [ 2, 3 ] ]
Test 'FactId' found following bins of size greater one:[ [ 2, 3 ] ]
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
gap> CalculateSiblings(128, false);
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
```

2.3.2 CalculateStrongSiblings

▷ `CalculateStrongSiblings(`*order[, verbose]*`)` (function)

Returns: A list of lists of ids of groups of size *order* that are strong siblings.

First calculats all siblings of size *order* via CalculateSiblings (2.3.1) and then determines all strong siblings among them.

Example

```
gap> CalculateStrongSiblings(256, false);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ], [ 1741, 1742 ], [ 3678, 3679 ],
  [ 4154, 4157 ], [ 4155, 4158 ], [ 4156, 4159 ] ]
```

2.4 Functions for building a search tree

This section describes all functions and variables connected to building a search tree that can be used with IdGroup512 (2.1.2) to identify groups via implemented invariants.

In short this is done be initializing a tree via calling TestOnClusterList (2.4.3) with a selected test on a list of (lists of) ids of groups of given order and then iteratnig the following three steps

- Collecting the clusters of the curent tree via ClusterList (2.4.1),
- Performing a choosen test on these clusters via TestOnClusterList (2.4.3),
- Extending the tree with these test results via AddClusterTestResults (2.4.4).

A very effective set of invaraints can be obtained by calling CocTestOnClusterList (2.4.6) in the second step. This should only be done when the groups of each provided cluster yields the same value when the test CocList is applied. Furthermore these invaraints should be used for as few as possible clusters since CocTestOnClusterList (2.4.6) includes a search for suitable parameters than can take quite some time.

2.4.1 ClusterList

▷ ClusterList(*tree*) (function)

Returns: A list of all clusters of groups in *tree* (i.e. the leafs of *tree*).

Iterates through *tree* and collects information about each leaf as a record. This includes the size of the leaf, the path through *tree* leading to the leaf, the tests performed including descriptions of performed powermap-tests and the ids of the groups contained in the leaf.

Example

```
gap> tree := rec( fp := [ 63049, 75721, 4310 ],  next := [ [ 3, 8 ], [ 4 ],
gap>            rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
gap>                test := "MaxSubId" ) ], test := "ElmOrds" );;
gap> ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 63049 ], size := 2,
       tests := [ "ElmOrds" ] ),
  rec( descl := [  ], ids := [ 4 ], path := [ 75721 ], size := 1,
       tests := [ "ElmOrds" ] ),
  rec( descl := [  ], ids := [ 5 ], path := [ 4310, 11744 ], size := 1,
       tests := [ "ElmOrds", "MaxSubId" ] ),
  rec( descl := [  ], ids := [ 6, 7 ], path := [ 4310, 86646 ], size := 2,
       tests := [ "ElmOrds", "MaxSubId" ] ) ]
```

2.4.2 TestOnGroup

▷ TestOnGroup(*G, test[, opt]*) (function)

Returns: The result of *test* applied to *G*, if desried encoded as an integer.
Performs the selected *test* on the group *G*. The following choices for *test* are implemented:

- *"ElmOrds"*: Collects the orders of all elements of *G*.
- *"Center"*: Calculates the abelian invariants of the center of *G*.
- *"Rank"*: Calculates the rank of *G*.
- *"AbelInv"*: Computes the order of each term of the derived series and the abelian Invariants of the derived factors of *G*.
- *"UCSId"*: Identifies the proper subgroups of the upper central series of *G* via IdGroup.
- *"LCSId"*: Identifies the proper subgroups of the lower central series of *G* via IdGroup.
- *"PCSId"*: Identifies the proper subgroups of the p-central series of *G* via IdGroup.
- *"SpSubId"*: Computes the Frattini subgroup, the upper and lower central series of *G* and identifies all terms if possible via IdGroup.
- *"MaxSubId"*: Collects the sizes and isomorphism types of all maximal subgroups of *G* via IdGroup.
- *"SubId"*: Collects the sizes and isormorphism types of conjugacy classes of all proper subgoups via IdGroup.
- *"MaxFactId"*: Collects the sizes and isomorphism types of all maximal factors of *G* via IdGroup.
- *"CentQuot"*: Collects the sizes and isomorphism types of all quotients of *G* with non-trivial elements of the center of *G* via IdGroup.
- *"LCSFactId"*: Collects the isomorphism types of all quotients of *G* with with cyclic subgroups generated by by non-trivial elements of the last non-trivial term of the p-central series.
- *"FactId"*: Identifies all proper factors of *G* via IdGroup.
- *"CocList"*: Collects the number of conjugacy classes of a *G* of same length and same representative order (the order of a representative).
- *"PMCycSt"*: Determines the cycle-structure invariant for the power map $x \mapsto x^3$ on the clusters of conjugacy classes via PowerMapCocCycSt.
- *"OutId"*: Tries to identify the groups of outer automorphisms of *G*, either via IdGroup (if available) or via its size and TestOnGroup with *"ElmOrds"* and *"CocList"* (*"MaxSubId"* and *"CentQuot"* can also be used if feasible).
- *"LattEqv"*: Calculates all conjugacy classes of proper subgoups of *G*, stores their size, identifies the the isomorphism types of each class and determines the isomorphism types and conjugacy classes in *G* of its maximal subgroups and minimal supergroups.

This function accepts a record *opt* of options with the single content *opt.pack*, a boolean that determines if the result of a test should be encoded as an integer. The default value of *opt* is *rec*(*pack* :=true).

Example

```
gap> G := SmallGroup(512, 1405);;
gap> TestOnGroup(G, "ElmOrds");
85416
gap> TestOnGroup(G, "ElmOrds", rec(pack := false));
[ [ 1, 1 ], [ 2, 7 ], [ 4, 24 ], [ 8, 288 ], [ 16, 64 ], [ 32, 128 ] ]
```

2.4.3 TestOnClusterList

▷ TestOnClusterList(*CL, test[, opt]*) (function)

Returns: Either a list of pairs each containg the path to the respective cluster (which is a leaf of a search tree) and the subbranch resulting from the results of *test*, or just the list of subbranches if *CL* contains only lists of integers. If *CL* consits of a single list of integers the single resulting subbranch is returned on its own.

Performs the choosen test on the groups of each record in the *CL*. The tests are performed by TestOnGroup (2.4.2). *CL* could either be a list of records produced by ClusterList (2.4.1) or be a list of lists containing ids. In the former case the output is a list of pairs containg the path of a cluster and the subbranch resulting from the test. In the latter case only a list of subbranches is returned. If in the latter case the returned list only one contains one subbranch, the subbranch itself is returned. This is useful when building the first level of a search tree. This function accepts a record *opt* of options with the following content:

- *pack*: A boolean that is handed to TestOnGroup (2.4.2).
- *ign1cl*: A boolean that determines if clusters of size 1 should be ignored.
- *order*: An integer giving the order of the groups with ids in *CL*.
- *verbose*: A boolean that determines if progress should be printed.
- *notetest*: A boolean that decides if the performed test should be noted in the returned subbranches. If so subbranches, that do not split a cluster are omited.

The default value of *opt* is *rec*(*pack*:=true, *ign*1*cl*:=true, *order*:=512, *verbose*:=true, *notetest*:=true).

Example

```
gap> TestOnClusterList([[1..10]],"ElmOrds", rec(verbose:=false));
rec( fp := [ 88570, 4310, 63049, 75721 ],
     next := [ [ 1 ], [ 2, 5, 6, 7, 9, 10 ], [ 3, 8 ], [ 4 ] ],
     test := "ElmOrds")
gap> TestOnClusterList([[1,2,5],[7,8,9]],"ElmOrds", rec(verbose:=false));
[ rec( fp := [ 88570, 4310 ], next := [ [ 1 ], [ 2, 5 ] ],
       test := "ElmOrds" ),
  rec( fp := [ 4310, 63049 ], next := [ [ 7, 9 ], [ 8 ] ],
       test := "ElmOrds") ]
gap> tree := TestOnClusterList([[3..8]],"ElmOrds", rec(verbose:=false));;
gap> CL := ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 63049 ], size := 2,
```

```
        tests := [ "ElmOrds" ]),
    rec( descl := [  ], ids := [ 4 ], path := [ 75721 ], size := 1,
        tests := [ "ElmOrds" ]),
    rec( descl := [  ], ids := [ 5, 6, 7 ], path := [ 4310 ], size := 3,
        tests := [ "ElmOrds" ]) ]
gap> TestOnClusterList(CL, "MaxSubId", rec(verbose:=false, notetest := false));
[ [ [ 63049 ], rec( fp := [ 62603 ], next := [ [ 3, 8 ] ] ) ],
  [ [ 4310 ], rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ] ) ] ]
```

2.4.4 AddClusterTestResults

▷ `AddClusterTestResults(`*tree, TR*`)` (function)

Returns: The extended *tree* with the subbranches in *TR*.

Extends *tree* by the test results stored in *TR*, a list of pairs each containing the path in *tree* and a branch (a tree of depth one), extending the corresponding cluster of the inital *tree*.

Example

```
gap> tree := TestOnClusterList([[3..8]],"ElmOrds", rec(verbose:=false));
rec( fp := [ 63049, 75721, 4310 ], next := [ [ 3, 8 ], [ 4 ], [ 5, 6, 7 ] ],
     test := "ElmOrds")
gap> TR:=TestOnClusterList(ClusterList(tree),"MaxSubId",rec(verbose:=false));
[ [ [ 4310 ], rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
                   test := "MaxSubId" ) ] ]
gap> AddClusterTestResults(tree,TR);
rec( fp := [ 63049, 75721, 4310 ],
     next := [ rec( fp := [ 62603 ], next := [ [ 3, 8 ] ] ), [ 4 ],
               rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
     test := "MaxSubId") ], test := "ElmOrds" )
```

2.4.5 BuildSearchTree

▷ `BuildSearchTree(`*N, order, tests*`)` (function)

Returns: A search tree (implemented as a record).

Builds a search tree for the groups with ids in *N* or, if *N* is an integer, the first *N* groups of size *order*, using `TestOnClusterList` (2.4.3) with the tests from the list *tests*.

Example

```
gap> BuildSearchTree(5,512,["ElmOrds", "MaxSubId"]);
rec( fp := [ 88570, 4310, 63049, 75721 ], next := [ [ 1 ],
     rec( fp := [ 76646, 11744 ], next := [ [ 2 ], [ 5 ] ],
     test := "MaxSubId" ), [ 3 ], [ 4 ] ], test := "ElmOrds" )
gap> BuildSearchTree([1,2,3,5,7], 512, ["ElmOrds", "CocList"]);
rec( fp := [ 88570, 4310, 63049 ], next := [ [ 1 ],
     rec( fp := [ 96561, 32173 ], next := [ [ 2 ], [ 5, 7 ] ],
     test := "CocList" ), [ 3 ] ], test := "ElmOrds" )
```

2.4.6 CocTestOnClusterList

▷ `CocTestOnClusterList(`*CL[, opt]*`)` (function)

Returns: A list of pairs each containing the path of the respective cluster and the subbranch resulting from `CocSplitRoutine`.

Tries to split each cluster in *CL* via tests on the lists of conjugacy classes. *CL* must be a cluster list produced by ClusterList (2.4.1). This function accepts a record *opt* of options with the following content:

- *pack*: A boolean that determines if the result should be encoded as an intger.
- *order*: An integer giving the order of the groups with ids in *CL*.
- *iterate*: An integer that determines the maximal number of iterations done while searching parameters.
- *verbose*: An boolean that determines if progress should be printed.

The default value of *opt* is *rec(pack*:=true, *order*:=512, *iterate*:=1,*verbose*:=true*)*.

Example

```
gap> tree := TestOnClusterList([[3,5,6,7,8]],"CocList",rec(verbose:=false));;
gap> CL := ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 337 ], size := 2,
       tests := [ "CocList" ] ),
  rec( descl := [  ], ids := [ 5, 6, 7 ], path := [ 32173 ], size := 3,
       tests := [ "CocList" ] ) ]
gap> TR := CocTestOnClusterList(CL,rec(verbose:=false));;
gap> AddClusterTestResults(tree, TR);;
gap> ClusterList(tree);
[ rec( descl := [ [ 302004 ] ], ids := [ 3 ], path := [ 337, 1363 ], size := 1,
       tests := [ "CocList", "" ]),
  rec( descl := [ [ 302004 ] ], ids := [ 8 ], path := [ 337, 46760 ], size := 1,
       tests := [ "CocList", ""  ]),
  rec( descl := [ [ 104003 ] ], ids := [ 5 ], path := [ 32173, 1214 ],
       size := 1, tests := [ "CocList", ""  ] ),
  rec( descl := [ [ 104003 ], [ 40204005 ] ], ids := [ 6 ],
       path := [ 32173, 16, 20792 ], size := 1, tests := [ "CocList", ""  ] ),
  rec( descl := [ [ 104003 ], [ 40204005 ] ], ids := [ 7 ],
       path := [ 32173, 16, 46881 ], size := 1, tests := [ "CocList", ""  ] ) ]
```

2.4.7 ParametersRandPresentTest

▷ ParametersRandPresentTest(*CL*, *order*, *maxl*) (function)

Returns: A list consisting of pairs of a list of ids and a record containing the parameters for a random presentation test via ID_GROUP_FUNCS[18].

Determines the needed parameters for the groups of size *order* in the clusters *cl* of *CL*. Creates the clusters of conjugcy classes and alters them accordingly to the *cl.descl*. Afterwards minimal generators for all groups and their position among the clusters of conjugacy classes. If the positions do not coincide, the positions of the generators of the first group are selected and the generators of the other groups are selected from the same clusters. Otherwise the positions and generators are sorted by the positions. Sequently the function tries to refine the corresponding clusters to limit the choices to choose from and repeat the process until either the number of refinments is equal or larger than *maxl* or no more refinments could be found.

Example

```
gap> ParametersRandPresentTest([rec(descl:=[], ids:=[1367, 1371])], 512, 0);
[ [ [ 1367, 1371 ], rec( descl := [  ],
```

```
      fp := [ 39080, 80814 ], func := 18, pos := [ 8, 10 ] ) ] ]
gap> ParametersRandPresentTest([rec(descl:=[], ids:=[1367, 1371])], 512, 1);
[ [ [ 1367, 1371 ], rec( descl := [ 106003 ],
      fp := [ 39080, 80814 ], func := 18, pos := [ 9, 11 ] ) ] ]
```

Index

AddClusterTestResults, 12

BuildSearchTree, 12

CalculateSiblings, 8
CalculateStrongSiblings, 8
ClusterList, 9
CocTestOnClusterList, 12

IdGroup512, 5
IdGroup512Exceptions, 6
ID_GROUP_512_TREE, 6
InfoId512, 5

LoadTotalTree, 6

ParametersRandPresentTest, 13

Siblings, 7
SiblingsIds, 7
StrongSiblings, 8
StrongSiblingsIds, 7

TestOnClusterList, 11
TestOnGroup, 10